Rapport de Mini-projet en C++

Princess Run

Rapport de Mini-projet en C++

Sujet : Application de jeu de cachette

Réalisé par :

- NAJWA IMMI
- GHIZLANE HMAMA

Numéros Apogées :

- NAJWA IMMI {N. Apogée : 2416417}
- GHIZLANE HMAMA {N. Apogée : 2416563}

Encadré par :

- PR. HANNANE RACHIDA

Dans le cadre du module de : Programmation orienté objet C++

Établissement : Université Cadi Ayad – Faculté des sciences Semlalia Marrakech

Département : Département d'Informatique

Année-universitaire : 2025/2026

Date de rendu : 23²/04/2026

1. Introduction

1.1. Contexte du projet

Ce projet a été réalisé dans le cadre d'un mini-projet de programmation C++ par Najwa IMMI et Ghizlane HMAMA. Il s'agit d'un jeu 2D de type "endless runner" développé avec la bibliothèque SFML (Simple and Fast Multimedia Library) version 3.0.2. Le jeu simule l'aventure d'une princesse cherchant à atteindre un palais en évitant des obstacles, avec une mécanique de progression par niveaux.

1.2. Présentation générale du jeu

PrincessRun est un jeu d'action-plateforme où le joueur contrôle une princesse qui court automatiquement vers la droite. Les contrôles sont simples :

- **Flèche haut** : Sauter pour éviter les obstacles aériens.
- **Flèche bas** : Se baisser pour éviter les obstacles hauts.

Le jeu propose :

4 niveaux progressifs avec augmentation de la difficulté (distance, vitesse d'apparition des obstacles).

3 vies (système de santé visuel via une barre).

Un chrono par niveau (30 secondes).

Une distance à parcourir pour atteindre le palais.

Un menu principal avec options "Jouer", "A propos", "Quitter".

Les écrans incluent : menu principal, jeu, victoire/défaite, et "A propos" avec description du jeu.

1.3. Objectifs pédagogiques

- Maîtriser la programmation orientée objet en C++ (classes, encapsulation, polymorphisme).
- Utiliser SFML pour la gestion des fenêtres, événements, graphismes, textures et polices.
- Implémenter une boucle de jeu avec delta-temps, physique (saut, gravité), collisions et caméra.
- Gérer la mémoire avec RAII (Resource Acquisition Is Initialization).
- Compiler avec CMake.

2. Spécification des besoins

2.1. Besoins Fonctionnels

2.1.1. Menu principal

Affichage d'un fond (background_menu.jpg) et trois options navigables (flèches haut/bas, Entrée pour sélection).

Options :

- Jouer (lance Game::run())
- A propos (fenêtre descriptive)
- Quitter

2.1.2. Déroulement d'une partie

Boucle principale : handleEvents(), update(), render().

Joueur court automatiquement ; saut/se baisser sur input clavier.

Génération procédurale d'obstacles (butterfly, bird, stone, log) espacés.

Caméra suit le joueur avec décalage ; fond en parallaxe.

Progression : distance vers palais, timer.

2.1.3. Système de vie et fin de partie

3 vies ; collision avec obstacle enlève 1 vie et rend invulnérable 1.5s (clignotement).

Fin si 0 vie, temps écoulé ou distance atteinte.

Menu fin de jeu :

- Rejouer
- Niveau suivant (si victoire)
- Quitter

2.1.4. Progression

2.2. Besoins non fonctionnels

- Framerate limité à 90 FPS pour fluidité.
- Résolution fixe 800x600.
- Gestion d'erreurs sur chargement assets (textures/polices).
- **Performance** : obstacles supprimés hors-écran.
- **Portabilité** : C++ moderne (C++11/C++17) avec SFML, compilable sous Windows (Visual Studio / MinGW) et Linux.
- **Maintenabilité** : code organisé par classes, séparation header/implémentation, constantes centralisées (Constants.h).
- **Robustesse** : gestion propre des ressources (textures, polices) et des erreurs de chargement.

2.3. Cas d'utilisation

- Acteurs : Joueur (humain), Système (jeu).

- UC1 : Lancer l'application → Menu principal s'affiche.
- UC2 : Démarrer une partie → initialisation et début du jeu.
- UC3 : Jouer (sauter/se baisser) → éviter obstacles.
- UC4 : Collision détectée → perdre une vie → éventuellement état blessé.
- UC5 : Vies épuisées → afficher écran Defeat → option Rejouer/Quitter (Retour menu).
- UC6 : Objectif atteint → afficher écran Victory → option Rejouer/Niveau suivant/Quitter (Retour menu).
- UC7 : Consulter écran À propos → afficher une description du jeu.

3. Structure des classes

3.1. Classe Constants (Constants.h)

- **Type** : fichier header de constantes (namespace global de constantes)
- **But** : centraliser paramètres du jeu (tailles, vitesses, constantes physiques)

Éléments :

- const int WINDOW_WIDTH = 800; → largeur de la fenêtre en pixels.
- const int WINDOW_HEIGHT = 600; → hauteur de la fenêtre en pixels.
- const int PLAYER_SPEED = 10; → vitesse de déplacement du joueur (utilisée dans Player).
- const int GAME_TIME_LIMIT = 30; → durée du niveau en secondes.
- const float MIN_OBSTACLE_SPACING = 150.f; → espacement minimum entre obstacles au spawn.
- const float JUMP_FORCE = 20.f; → force de saut (déclarée mais non utilisée directement dans le code fourni).
- const float GRAVITY = 0.6f; → gravité (idem).
- const float GROUND_Y = 400.f; → position Y du sol (utilisée pour calculs de saut).
- const float BASE_PALACE_DISTANCE = 10000.f; → distance de base vers le palais (objectif).

3.2. Classe Game (Game.h / Game.cpp)

- **Type** : classe centrale qui orchestre le jeu.
- **Rôle** : gestion de la fenêtre, boucle principale, mise à jour des entités, spawn d'obstacles, gestion du temps, UI de fin de partie, caméra, etc.

Attributs (privés, extraits de Game.h) :

- sf::RenderWindow window; →fenêtre SFML.

- sf::View camera; →vue/caméra suivie.
- Player player; →instance du joueur.
- std::vector<std::unique_ptr<Entity>> obstacles; →conteneur polymorphique d'obstacles/entités (concrètement Obstacle).
- sf::Clock clock; →utilisé pour le chronomètre (décrément timeRemaining chaque seconde).
- int timeRemaining; →timer de niveau (s).
- float distanceRemaining; → distance restante vers le palais.
- float lastObstacleSpawnX; → position X du dernier spawn (contrôle espacement).
- float OBSTACLE_SPACING = 400.f; → espacement par défaut (membre initialisé).
- **Niveaux :**
    - int currentLevel;
    - int maxLevel;
    - float obstacleSpawnRate;
    - float playerSpeed;
    - int levelTimeLimit;
- **Backgrounds / UI :**
    - sf::Sprite backgroundSprite;
    - sf::Sprite palaceSprite;
    - sf::Font font;
    - sf::Text timerText;
    - sf::Text distance; (probablement un label, variable nommée distance)
    - sf::Text distanceText;
    - float initialDistance;
- **End menu / messages :**
    - sf::Text endMessage;
    - std::vector<sf::Text> menuOptions;
    - int menuIndex;
    - bool gameOver;
    - bool wonLevel;
- **Victoire/Défaite :**
    - sf::Sprite victorySprite;

- sf::Sprite defeatSprite;

Méthodes publiques :

- Game(); →constructeur : initialise fenêtre, charge textures via des getters statiques, configure textes, place palais, configure timer, etc.
- void run(); → boucle principale : appelle handleEvents(), update(), render() jusqu'à fermeture.
- void handleEvents(); →traite événements SFML (fermeture, navigation menu, entrées joueur via player.handleInput()).
- void update(); →logique par frame :
  - décrémente timeRemaining chaque seconde.
  - spawn d'obstacles si le joueur a avancé suffisamment.
  - itère obstacles, appelle update(), vérifie collision (player.getBounds().findIntersection(obstacle->getBounds())) et applique player.takeDamage() puis efface l'obstacle si collision ou hors écran.
  - met à jour caméra et background (parallax léger).
  - met à jour distanceRemaining et gère fin de niveau (victoire/défaite).
- void render(); →dessine la scène (si gameOver affiche menu de fin, sinon vue de jeu : background, palace, player, obstacles, UI).
- void spawnObstacle(); →génère un nouvel obstacle. Implémentation actuelle :
  - sélectionne aléatoirement le type parmi {"butterfly","bird","stone","log"}.
  - calcule yPos en fonction du type (sol pour stone/log, milieu/haut pour oiseaux/flies).
  - calcule xPos = player.getBounds().position.x + WINDOW_WIDTH + MIN_OBSTACLE_SPACING.
  - crée obstacle polymorphiquement : obstacles.emplace_back(std::make_unique<Obstacle>(type, xPos, yPos));
- void resetLevel(); →réinitialise l'état du niveau (score, timer, obstacles, position joueur, caméra).
- void initEndMenu(); → (re)génère les options du menu final selon victoire/défaite.

Remarques sur l'implémentation :

- Les collisions sont testées via sf::FloatRect::findIntersection entre player.getBounds() et obstacle->getBounds().
- Les obstacles sont stockés en tant que std::unique_ptr<Entity> : cela permet d'utiliser la hiérarchie et d'ajouter d'autres sous-classes d'Entity plus tard si nécessaire.
- spawnObstacle() laisse la logique d'apparition aléatoire telle quelle (tu peux personnaliser taux, types, ou ajouter entités décoratives).

3.3. Classe Menu (Menu.h / Menu.cpp)

- **Type** : classe concrète UI pour menu principal.
- **Rôle** : gérer l'affichage et la sélection des options du menu.

Attributs (privés) :

- sf::Font font; →police pour textes.
- std::vector<sf::Text> options; →liste des options affichées.
- int selectedIndex; →index option actuellement sélectionnée.
- sf::Sprite menuSprite; →background/menu image.

Constructeur :

- Menu(float width, float height); →initialise le menuSprite (texture background_menu.jpg), charge police GROBOLD.ttf, crée trois labels ("Jouer", "A propos", "Quitter") et positionne les textes.

Méthodes :

- void draw(sf::RenderWindow& window); →dessine menuSprite puis toutes les options.
- void moveUp(); →décrémente selectedIndex (et met à jour couleur des textes).
- void moveDown(); →incrémente selectedIndex.
- int getSelectedIndex(); →retourne l'index courant.

3.4. Classe Obstacle (Obstacle.h / Obstacle.cpp)

- **Type** : classe concrète, hérite de Entity.
- **Rôle :** représentation d'un obstacle réel (sprite, position, collision).

Attributs (privés) :

- sf::Sprite sprite; →sprite SFML qui contient la texture/transform du visuel de l'obstacle.

Constructeur :

- Obstacle(std::string type, float x, float y);
    - type : nom de l'image (par ex. "stone", "bird", "log", "butterfly" ou "arbre" si icône présente).
    - x, y : position initiale.

Méthodes (implémentées / override) :

- void update() override; →logique de mise à jour (dans la version actuelle : obstacles fixes, donc pas de déplacement).
- void draw(sf::RenderWindow& window) override; →dessine sprite sur la fenêtre.

- sf::FloatRect getBounds() override; →retourne sprite.getGlobalBounds() pour gestion des collisions.

Implémentation notable :

- Obstacle.cpp contient une fonction utilitaire statique getObstacleTexture(const std::string& type) qui utilise un cache static std::map<std::string, sf::Texture> et loadTexture("assets/" + type + ".png") pour charger/retourner la texture.
- Le constructeur initialise sprite avec la texture renvoyée, positionne et redimensionne (sprite.setScale({0.2f, 0.2f}); etc).

Rôle fonctionnel :

- Obstacles sont des entités collidables par défaut (Entity::isCollidable() retourne true), stockées polymorphiquement comme std::unique_ptr<Entity>.

3.5. Classe Player (Player.h / Player.cpp)

- **Type** : classe concrète représentant le joueur.
- **Rôle :** gérer état visuel, input, physique basique de saut, santé et UI du joueur.

Attributs (privés) :

sf::Texture texture; → (champ présent mais l'implémentation utilise des textures statiques internes pour différentes animations).

sf::Sprite sprite; →sprite du personnage.

sf::RectangleShape healthBarBackground; →rectangle fond de la barre de vie.

sf::RectangleShape healthBar; →rectangle représentant la vie restante.

sf::Clock invulnClock; →temporisateur pour période d'invulnérabilité.

bool invulnerable; →true si invulnérable après avoir pris un dégât.

float verticalVelocity; →vitesse verticale (saut).

bool isJumping; →true si en l'air.

bool isDucking; →true si accroupi.

float groundY; →Y de référence du sol pour le joueur.

int health; →points de vie (3 par défaut).

Constructeur :

Player(); →initialise sprite (texture de course), position initiale, barre de vie, etc.

Méthodes publiques :

- void handleInput(); →lit le clavier (Up pour sauter, Down pour s'accroupir) et met à jour flags.
- void setTexture(const sf::Texture& texture); →setter pour la texture (déclaré).

- void update(); →logique de physique (saut, gravité approximative), animation (changer texture selon état), déplacement horizontal constant, gestion invulnérabilité.
- void draw(sf::RenderWindow& window); →dessine le sprite du joueur.
- void drawUI(sf::RenderWindow& window); →dessine la barre de vie.
- sf::FloatRect getBounds(); →retourne sprite.getGlobalBounds() pour collision.
- void takeDamage(); →réduit health si pas invulnerable, applique état invulnérable et démarre invulnClock, met à jour healthBar.
- bool isAlive(); →retourne health > 0.
- void resetHealth(); →réinitialise health à 3 et l'UI.
- void resetPosition(); →repositionne le joueur à l'état initial (ajouté).

Détails d'implémentation :

- Textures run/jump/duck sont chargées via fonctions statiques qui appellent loadTexture("assets/...") (ex. getRunTex()).
- Le déplacement horizontal est géré par sprite.move({PLAYER_SPEED * 0.5f, 0.f}); →la caméra est gérée côté Game.
- L'invulnérabilité fait clignoter le sprite et dure 1.5 s.

3.6. Classe Utils (Utils.h / Utils.cpp)

- **Type** : fonctions utilitaires liées au chargement de ressources.

Déclarations (Utils.h) :

- sf::Texture loadTexture(const std::string& filename);
- sf::Font loadFont(const std::string& filename);

Implémentation (Utils.cpp) :

- sf::Texture loadTexture(const std::string& filename);
    - Crée sf::Texture texture; puis texture.loadFromFile(filename). En cas d'échec écrit std::cerr.
    - Retourne la texture par valeur.
- sf::Font loadFont(const std::string& filename);
    - Similaire : sf::Font font; font.openFromFile(filename) puis retour.

Rôle :

- Centraliser le chargement des ressources et messages d'erreur. (Remarque : fonctions retournent les objets par valeur, et certains usages dans le code mettent ces textures dans des variables/statics.)

3.7. Classe Entity (Entity.h)

- **Type :** classe abstraite (interface) pour entités graphiques / obstacles.
- **Rôle** : définir l'API commune pour tout ce qui peut être mis à jour / dessiné / testé en collision.

Méthodes (virtuelles) :

- virtual ~Entity() = default; →destructeur virtuel.
- virtual void update() = 0; →méthode de mise à jour par frame (pure virtual).
- virtual void draw(sf::RenderWindow& window) = 0; →dessiner l'entité (pure virtual).
- virtual sf::FloatRect getBounds() = 0; →obtenir la bounding box pour collisions (pure virtual).
- virtual bool isCollidable() const { return true; } →indique si l'entité doit être testée pour collision (par défaut true). Peut être surchargée (ex : nuages décoratifs).

Description / rôle :

- Fournit un contrat pour les classes concrètes (Obstacle, etc.). Permet stockage polymorphique (std::unique_ptr<Entity>).

3.8. Hiérarchie Obstacle

1) Entity (fichier : Entity.h)

- **Type** : classe abstraite (interface).
- **Rôle** : contrat commun pour toutes les entités dessinables / updatables / testables en collision.

Méthodes clés (publiques) :

- virtual ~Entity() = default;
  →destructeur virtuel.
- virtual void update() = 0;
  →mise à jour de l'entité chaque frame (pure virtual).
- virtual void draw(sf::RenderWindow& window) = 0;
  →dessiner l'entité (pure virtual).
- virtual sf::FloatRect getBounds() = 0;
  →renvoyer le rectangle de collision / bounding box (pure virtual).
- virtual bool isCollidable() const { return true; }
  →indique si l'entité doit être testée pour collision (valeur par défaut true ; peut être surchargée pour entités décoratives).

Particularités :

- Aucun état stocké ici : Entity définit l'API pour le polymorphisme.

- Permet de stocker des objets concrets dans des std::unique_ptr<Entity>.

2) Obstacle (fichiers : Obstacle.h, Obstacle.cpp)

- **Type** : classe concrète dérivée de Entity
  Déclaration importante : class Obstacle : public Entity.
- **Rôle** : implémentation d'un obstacle réel du jeu (sprite, position, détection de collision).

Attributs (privés) :

  - sf::Sprite sprite; →sprite SFML qui contient la texture et la transformation (position/scale).

Constructeur :

  - Obstacle(std::string type, float x, float y);
    →charge la texture correspondant à type (ex. "stone","bird","log","butterfly", …) et initialise le sprite à la position (x,y).

Méthodes clés (override) :

  - void update() override;
    →logique de mise à jour (dans le code actuel les obstacles sont statiques ; méthode prévue pour comportement dynamique si nécessaire).
  - void draw(sf::RenderWindow& window) override;
    →dessine le sprite sur la fenêtre.
  - sf::FloatRect getBounds() override;
    →retourne sprite.getGlobalBounds() (utilisé pour la détection de collision).

Particularités d'implémentation :

  - Les textures sont récupérées via une fonction utilitaire statique getObstacleTexture(const std::string& type) qui utilise un cache statique std::map<std::string, sf::Texture> et loadTexture("assets/" + type + ".png"). Ainsi une même texture n'est pas rechargée plusieurs fois.
  - Le sprite est initialisé avec la texture (souvent dans la liste d'initialisation du constructeur, ex. : sprite(getObstacleTexture(type))), puis mis à l'échelle (sprite.setScale(...)) et positionné (sprite.setPosition(...)).
  - Obstacle n'override pas isCollidable() ⇒ elle est collidable par défaut.
  - Conçu pour être utilisé via un pointeur polymorphique (std::unique_ptr<Entity>), ce qui facilite l'extension future (ajout d'autres sous-classes d'Entity).

Utilisation pratique dans le projet

- **Stockage** : les obstacles sont stockés dans Game sous la forme :
  - std::vector<std::unique_ptr<Entity>> obstacles;
- **Création (extrait)** : dans Game::spawnObstacle() on crée un obstacle concret et on l'insère polymorphiquement, par exemple :

    - obstacles.emplace_back(std::make_unique<Obstacle>(type, xPos, yPos));
- Collisions : la boucle d'update du Game appelle pour chaque obstacle :
    - (*it)->update();
    - puis teste la collision via player.getBounds().findIntersection((*it)->getBounds()) et, en cas d'intersection, appelle player.takeDamage() et efface l'obstacle.
- Rendu : dans Game::render() on parcourt obstacles et on appelle obs->draw(window);.

3.9. CMakeLists.txt

- Liste des sources/headers (modifiée au fil des changements).
- find_package(SFML 3.0 REQUIRED COMPONENTS Graphics Window System Audio) puis target_link_libraries(... SFML::Graphics SFML::Window SFML::System).

4. Relations entre Classes et Paradigme Orienté Objet

4.1. Héritage

Le projet utilise l'héritage pour factoriser le comportement commun des objets « entité » du jeu.

- **Classe de base** : Entity (Entity.h) →interface abstraite qui définit le contrat commun :
    - Méthodes virtuelles pures : update(), draw(sf::RenderWindow&), getBounds().
    - Méthode virtuelle non pure : isCollidable() (valeur par défaut true).
- **Sous-classe** : Obstacle (Obstacle.h / Obstacle.cpp) hérite de Entity et implémente les méthodes :
    - update() override, draw(...) override, getBounds() override.
- **Rôle de l'héritage** :
    - Factorisation du code commun aux objets gérés par la boucle de jeu (mise à jour, rendu, collision).
    - Permet de stocker et manipuler des instances concrètes via le type abstrait Entity, facilitant l'extension (ajout futur d'autres types d'entités).

Exemple concret dans le code :

- std::vector<std::unique_ptr<Entity>> obstacles; dans Game permet de conserver différents types d'entités derrière l'interface Entity.

4.2. Polymorphisme

Le polymorphisme dynamique est utilisé pour appeler les bonnes implémentations au runtime via des pointeurs intelligents.

- Stockage polymorphique :
    - std::unique_ptr<Entity> pointe vers des objets concrets (Obstacle).

- **Appels virtuels** :
  - Dans la boucle d'update du Game, le code appelle (*it)->update(), (*it)->getBounds() et (*it)->draw(window) sans connaître le type concret de l'entité. L'appel dispatché au runtime exécute l'implémentation correcte (Obstacle::update, Obstacle::draw, ...).
- **Avantages** :
  - Découplage entre l'usage (logiciel de jeu) et les implémentations concrètes.
  - Possibilité d'ajouter des entités décoratives non collidables ou des obstacles spécialisés sans changer la boucle de gestion dans Game.

4.3. Encapsulation

L'encapsulation est respectée : les données internes des classes sont cachées et manipulées via des méthodes publiques.

- **Exemples** :
  - Player (Player.h) : attributs privés (sprite, healthBar, health, isJumping, ...) et méthodes publiques (handleInput(), update(), draw(), takeDamage(), getBounds(), resetHealth()).
  - Obstacle : sprite est privé ; interaction via getBounds(), update() et draw().
  - Menu : police et options sont privées, l'interface publique offre moveUp(), moveDown(), draw().
- **Bénéfices de l'encapsulation :**
  - Protection de l'état interne (prévention des modifications directes indésirables).
  - Interface claire et stable pour les autres composants (Game utilise les méthodes publiques sans accéder aux membres internes).
- **Bonne pratique observée :**
  - Les classes exposent des opérations utiles (ex. Player::takeDamage() gère à la fois la décrémentation du HP et l'invulnérabilité), évitant la logique dispersée.

4.4. Gestion de la mémoire et RAII

Le projet applique des principes de gestion de ressources en C++ et RAII (Resource Acquisition Is Initialization), avec quelques points d'attention.

- **Utilisation de RAII / objets gérés** :
  - SFML (sf::Texture, sf::Sprite, sf::Font, sf::Clock) utilise RAII : les ressources sont acquises lors de la construction et libérées automatiquement à la destruction.
  - std::unique_ptr<Entity> est utilisé pour la gestion des entités polymorphiques : destruction automatique des objets quand le vecteur efface les éléments ou quand il sort de portée.

- **Gestion des collections :**
    - std::vector<std::unique_ptr<Entity>> obstacles; évite les fuites mémoire liées aux pointeurs bruts et facilite le transfert de propriété.
- **Chargement des ressources** :
    - Utils::loadTexture() et Utils::loadFont() retournent des objets sf::Texture et sf::Font. Le projet utilise aussi des caches statiques (ex. std::map<std::string, sf::Texture> textureCache; dans Obstacle.cpp) pour réutiliser les textures et éviter des recharges répétées.
- **Points à surveiller / suggestions** :
    - Retourner sf::Texture par valeur est acceptable, SFML gère les copies/moves, mais il faut s'assurer que les sf::Texture vivants restent accessibles tant que des sf::Sprite les utilisent. L'approche de cache statique est correcte pour assurer la durée de vie des textures.
    - Eviter les cycles de dépendance ou les pointeurs bruts ; unique_ptr et les objets RAII couvrent la majorité des cas.
    - Pour des ressources globales (textures/polices), utiliser explicitement un gestionnaire de ressources (ResourceManager) rendent le code plus explicite et testable.
- **Exemple d'usage correct :**
    - Lors de l'effacement d'un obstacle : it = obstacles.erase(it); →le unique_ptr est détruit automatiquement et libère l'objet Obstacle.

5. Environnement de travail

5.1. Outils et technologies

Langage : C++ (standard C++17).

Système de build : CMake (fichier CMakeLists.txt, version minimale requise : 3.16).

Bibliothèques :

SFML 3.x (Graphics, Window, System, Audio) →la copie locale SFML-3.0.2 est présente dans le dépôt.

Compilateur : compilateur C++ compatible (MinGW-w64 ou MSVC sous Windows selon la configuration CMake / IDE). Le projet a des builds pour MinGW et pour des configurations CMake IDE (ex. cmake-build-debug-mingw, cmake-build-debug).

Environnement / OS : développé sur Windows (PowerShell utilisé comme shell dans l'environnement fourni).

Éditeur / IDE : projet compatible avec IDEs C++ (ex. CLion, Visual Studio, VS Code). Les dossiers cmake-build-debug indiquent usage possible de CLion.

Outils auxiliaires :

PowerShell pour exécuter les commandes CMake / build.

SFML fourni localement (dossier SFML-3.0.2) pour faciliter l'exécution sans installation système.

Remarque : le chargement des ressources (images, polices) se fait depuis le dossier assets. Pour exécuter correctement l'application, le répertoire de travail doit permettre l'accès relatif à assets (soit via copies dans le dossier de build, soit en lançant l'exécutable depuis la racine du projet).

5.2. Architecture des fichiers

Arborescence principale (extrait et rôle) :

- CMakeLists.txt
  Fichier de configuration CMake qui liste les sources, headers, configure le standard C++ et lie SFML.
- **Sources / headers principaux** :
  - main.cpp — point d'entrée du programme (initialise Game et lance la boucle).
  - Game.h / Game.cpp — classe contrôlant la boucle de jeu, la fenêtre, la caméra, le spawn et la gestion des entités, timer et état de niveau (game over / victoire).
  - Menu.h / Menu.cpp — gestion du menu principal (affichage, navigation).
  - Player.h / Player.cpp — représentation du joueur : sprite, entrées clavier, physique basique (saut), gestion de la vie et UI (barre de vie).
  - Obstacle.h / Obstacle.cpp — classe concrète d'obstacle (hérite de Entity) : sprite, position, bounding box pour collisions, cache de textures.
  - Entity.h — interface abstraite (contrat) pour les entités du jeu : update(), draw(), getBounds(), isCollidable().
  - Utils.h / Utils.cpp — fonctions utilitaires pour charger textures et polices (loadTexture, loadFont) avec gestion d'erreur simple.
  - Constants.h — constantes globales du jeu (dimensions fenêtre, vitesses, gravité, constants de gameplay).
- **Ressources / médias** :
  - assets →dossier contenant toutes les ressources : images (.png), fonds (.jpg), polices (.ttf) utilisées par le jeu. Exemples :
    - background.jpg, background_menu.jpg, palace.png, arbre.png, nuage.png, pri ncess_run.png, GROBOLD.ttf, etc.
  - **Note :** certaines copies des assets sont présentes aussi dans les dossiers de build (assets) pour pouvoir exécuter l'exécutable créé par l'IDE.
- **Dossiers de build et utilitaires** :
  - build →dossier de build CMake (ex. généré via cmake -B build).

- cmake-build-debug, cmake-build-debug-mingw →sorties de build (configurations IDE / toolchain). Contiennent l'exécutable PrincessRun.exe, fichiers CMake générés et éventuellement copies des assets.
- SFML-3.0.2 →distribution locale de SFML (bin, include, lib, etc.) fournie dans le dépôt.

Résumé des responsabilités par fichier-clé :

- Game : orchestration générale (boucle, spawn, collisions, UI fin).
- Player : état et comportement du joueur (input, déplacement, saut, santé).
- Obstacle : visuel + collision des obstacles ; utilise cache de textures et Entity comme base.
- Entity : abstraction pour les objets gérés par la boucle (facilite polymorphisme).
- Utils : centralisation du chargement de ressources.
- Constants : paramètres globaux modifiables pour équilibrage.

6. Interfaces du jeu

6.1. Menu principal

But : permettre au joueur de lancer le jeu, d'accéder à l'écran « À propos » ou de quitter.

Éléments affichés :

Image de fond (background_menu.jpg) rendue via un sf::Sprite.

Liste d'options textuelles : « Jouer », « A propos », « Quitter » (vecteur std::vector<sf::Text>).

Option sélectionnée mise en évidence (couleur différente).

Contrôles :

Flèches Haut/Bas pour changer la sélection (moveUp() / moveDown()).

Entrée (Enter) pour valider l'option sélectionnée.

Comportement : le menu est dessiné avant de lancer la boucle de jeu ; sélecteur visuel et navigation clavier gérés dans Menu et déclenchés par Game::handleEvents().

Figure 1 *: Interface du menu principale {Jouer, A propos, Quitter}*

6.2. Ecran à propos

Emplacement / état : option du menu principal (label existant dans Menu.cpp).

Rôle : afficher des informations sur le projet / les auteurs (dans la version courante, il peut s'agir d'un écran simple ou d'un placeholder selon implémentation).

Présentation recommandée : texte centré avec la police du jeu (GROBOLD.ttf)

Figure 2 *: Interface A props {Description + Contôle clavier}*

6.3. Interface de jeu

6.3.1. Partie en cours

Logic : Game::update() met à jour joueur, obstacles, chrono et calcule distanceRemaining.

Spawn d'obstacles : spawnObstacle() ajoute des obstacles devant le joueur en fonction de sa progression.

rendu : scène 2D avec parallax léger, sprites et HUD textuels.

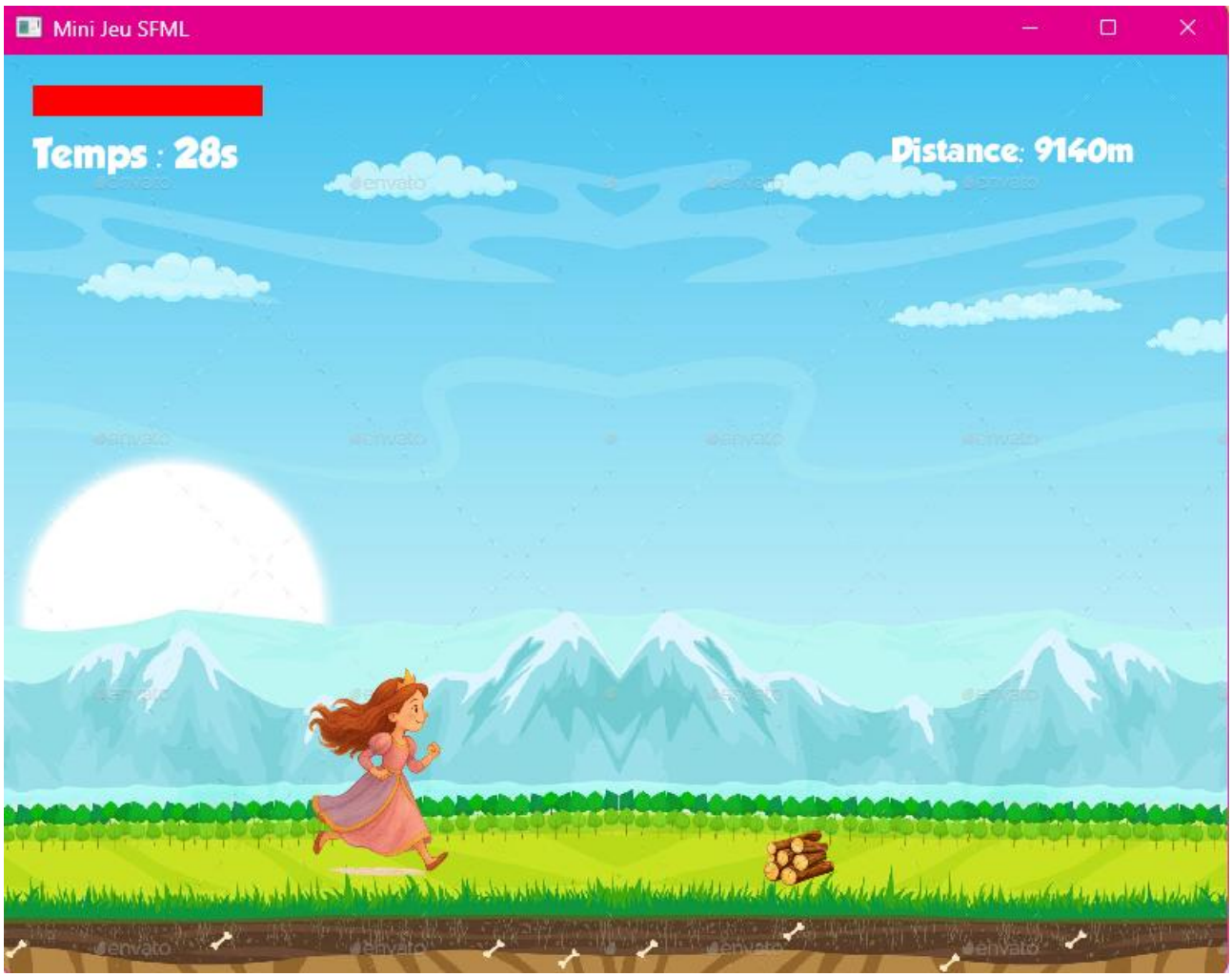

Figure 3 *: Interface du jeu en cours {Temps : 28s , Distance : 9140m , 3 vies}*

6.3.2. Joueur blessé

Indicateur visuel : lorsque le joueur prend un dégât (Player::takeDamage()), le sprite clignote (couleur / transparence) pendant une période d'invulnérabilité (invulnClock).

Effet sur UI : la barre de vie (healthBar) est réduite proportionnellement au nombre de vies restantes.

Comportement système : si player.takeDamage() réduit health à 0, Game::update() détecte !player.isAlive() et bascule en écran de fin (défaite).

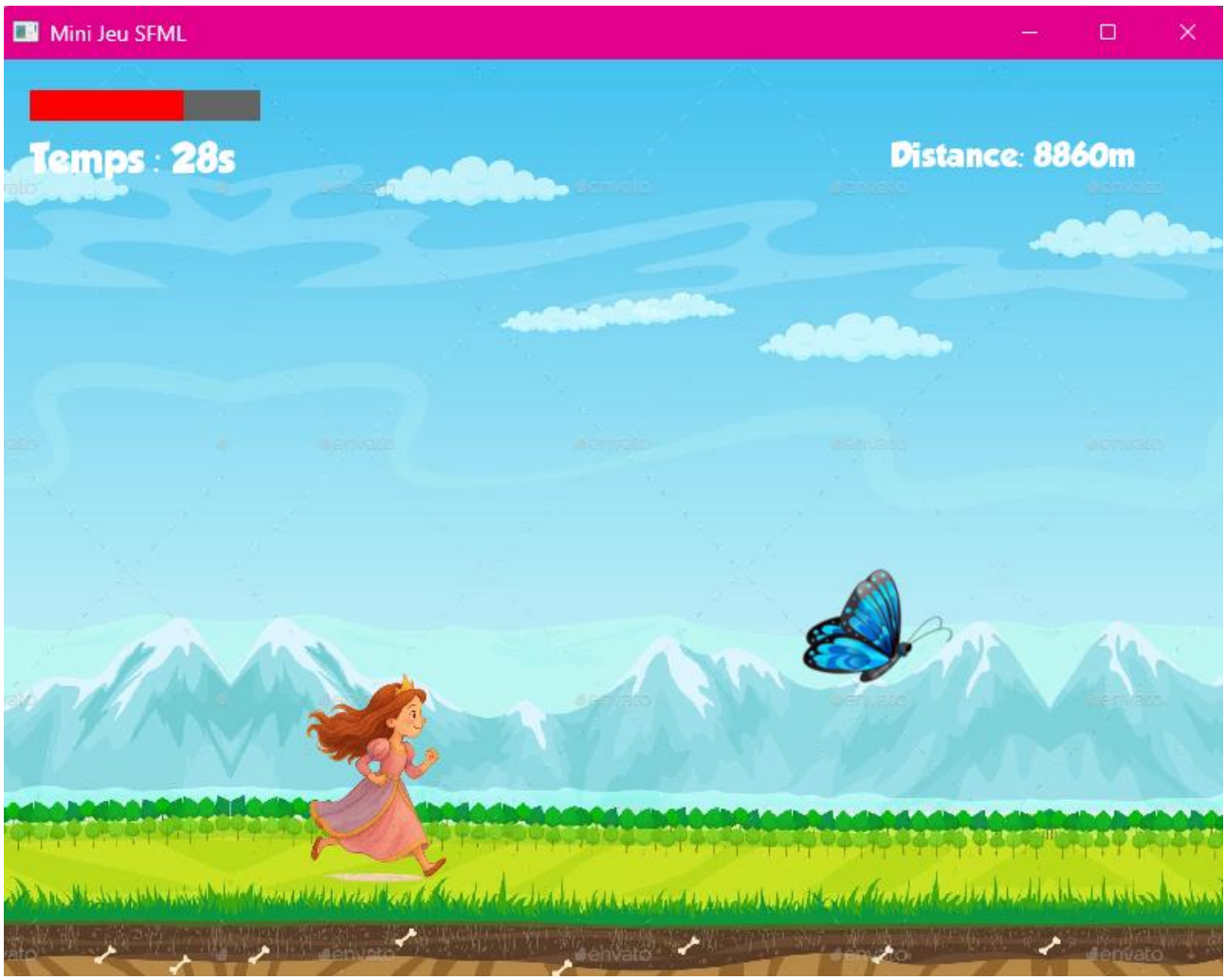

Figure 4 *: Interface joueur blessé {il reste 2 vies}*

6.3.3. Joueur sauté

Contrôles : le saut est déclenché par la touche Flèche Haut (Up).

Comportement visuel : quand le joueur saute, sa texture passe de l'animation de course à la texture de saut (getJumpTex() dans Player), ce qui change l'apparence du sprite pendant la phase aérienne.

Implémentation (où) :

Détection de l'entrée : Player::handleInput() (vérifie la touche Up et l'état !isJumping pour initier un saut).

État : flag isJumping = true et impulsion initiale verticalVelocity = -20.f (valeur actuelle dans le code).

Mise à jour : Player::update() applique chaque frame verticalVelocity += 1.f; puis sprite.move({0.f, verticalVelocity});. Quand la position Y atteint ou dépasse groundY, le joueur est reposé au sol (isJumping = false, verticalVelocity = 0).

Collision / hitbox :

La hitbox utilisée est sprite.getGlobalBounds() ; elle suit automatiquement la position du sprite en vol (hitbox dynamique).

Pendant le saut, la hitbox se déplace verticalement avec le sprite, ce qui modifie en temps réel la zone de collision.

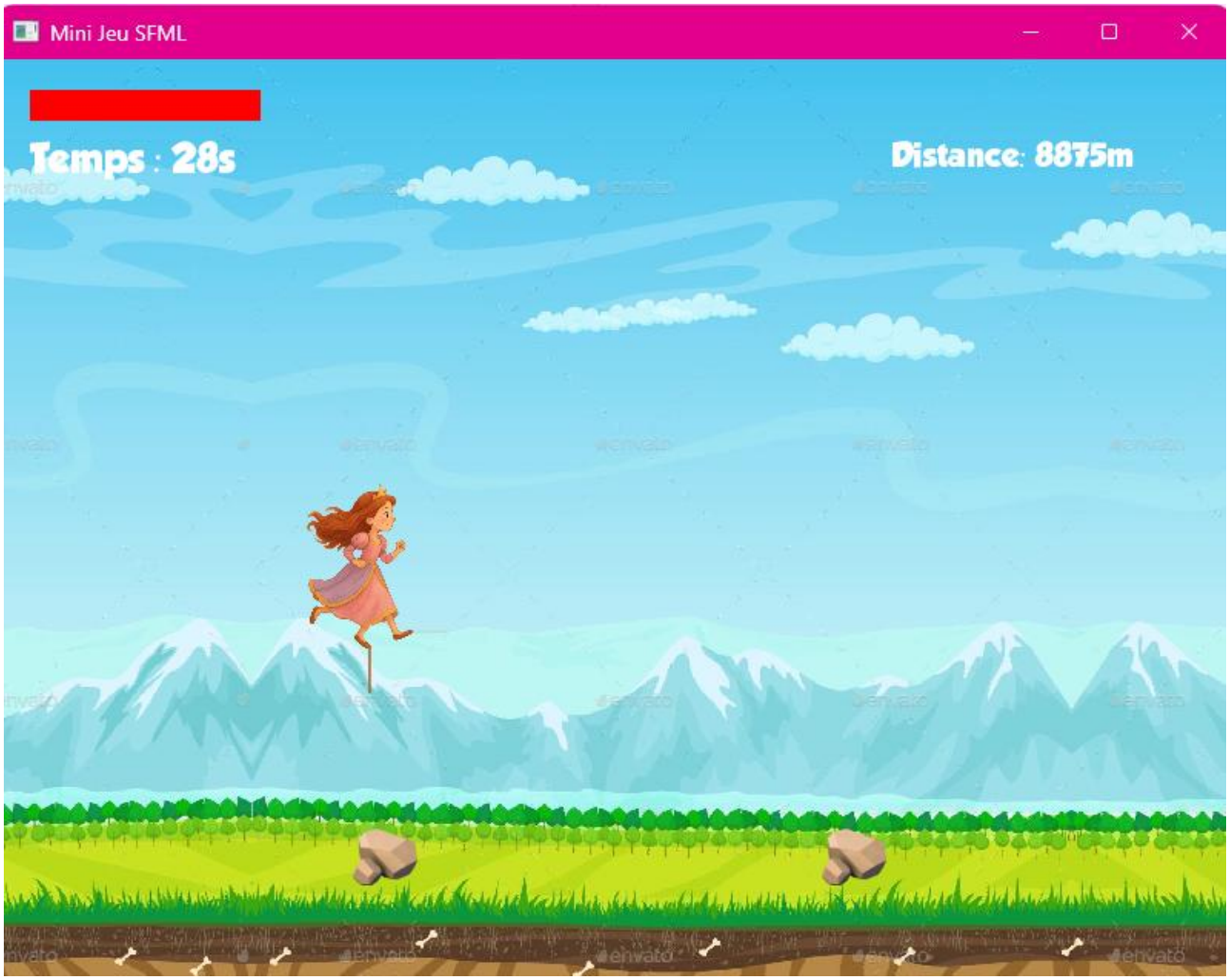

Figure 5 *: Interface joueur sauté {feléche haut}*

6.3.4. Joueur se baissé

Contrôles : le joueur se baisse en maintenant la touche Flèche Bas (Down).

Comportement visuel : en position accroupie, la texture du joueur passe à l'image de duck (getDuckTex()), et le sprite est repositionné légèrement vers le bas pour simuler l'accroupissement (sprite.setPosition({currentX, groundY + 60.f}) dans le code).

Implémentation (où) :

Player::handleInput() met isDucking = true si Down est pressé et que le joueur n'est pas en train de sauter.

Player::update() applique la texture de duck et ajuste la position verticale du sprite uniquement si !isJumping (le joueur ne peut pas s'accroupir en plein saut selon l'implémentation actuelle).

Collision / hitbox :

Comme pour le saut, la hitbox suit sprite.getGlobalBounds(). En position accroupie la hauteur du sprite (et donc de la hitbox) change, réduisant la zone verticale de collision (utile pour passer sous certains obstacles).

Important : l'ajustement de position et d'échelle modifie la hitbox ; il faut s'assurer que l'offset appliqué au sprite correspond au collider désiré (éviter que la hitbox glisse visuellement hors du sprite).

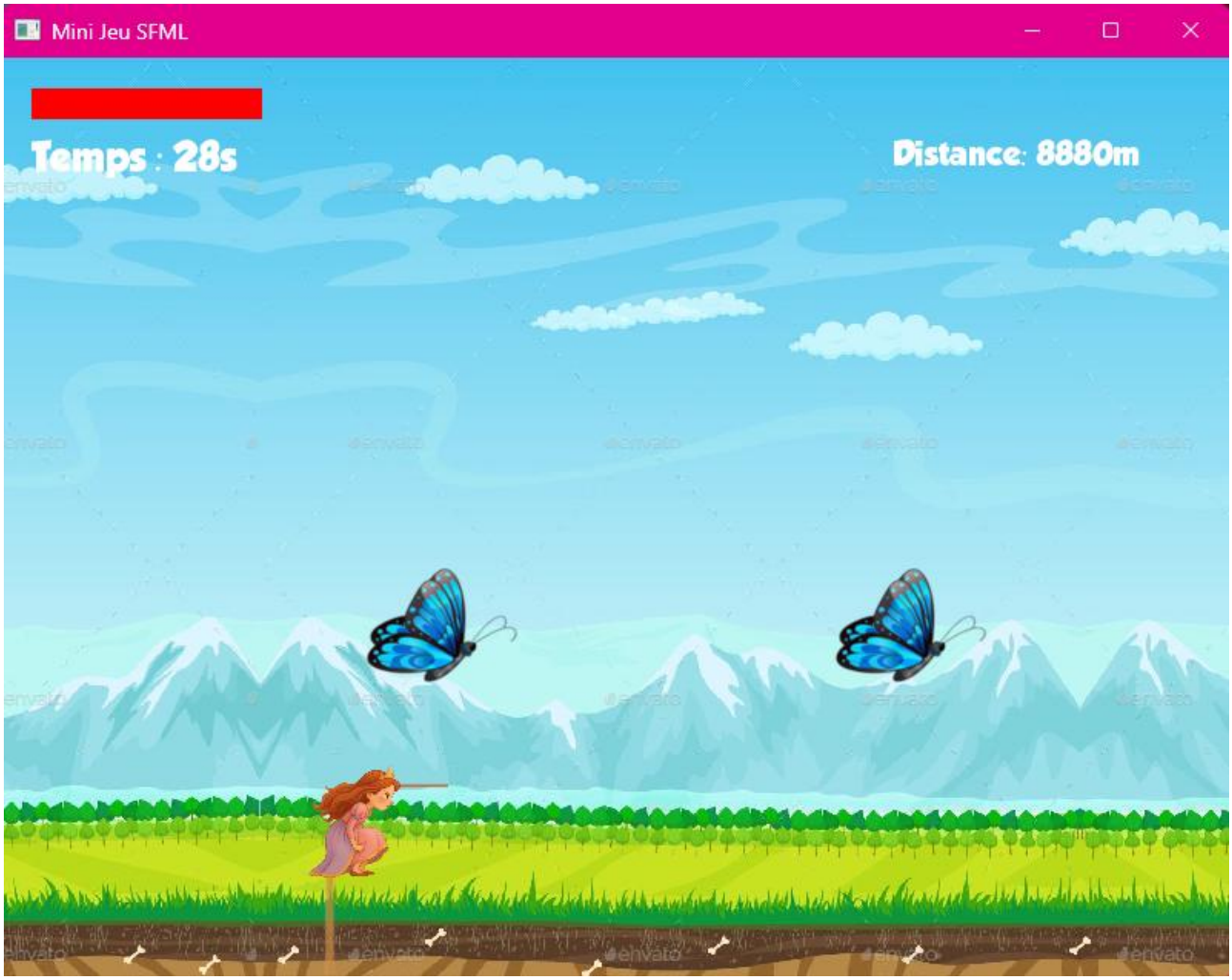

Figure 6 *: interface joueur se baissé {fléche bas}*

6.4. Ecran Defeat

Présentation :

Fond d'écran de défaite : defeatSprite (texture defeat_background.jpg).

Message de défaite : endMessage (texte explicite).

Menu d'options : menuOptions (ex. « Rejouer », « Quitter »).

Navigation :

Flèches Haut/Bas pour naviguer entre les options.

Entrée pour sélectionner : « Rejouer » appelle resetLevel(), « Quitter » ferme la fenêtre.

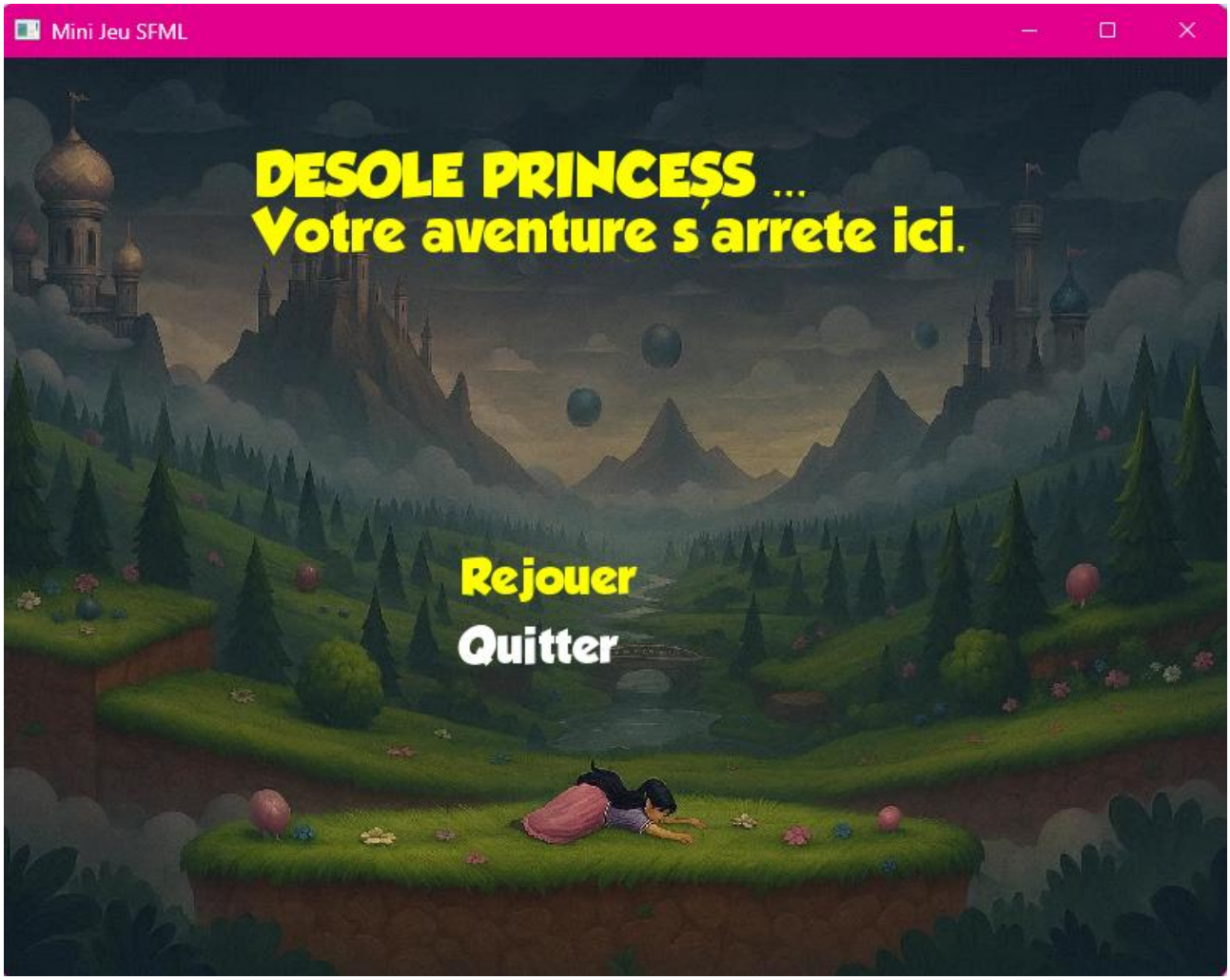

***Figure** 7 : Interface de défaite {Message + options rejouer ou quitter}*

6.5. Ecran Vectory

Présentation :

Fond d'écran de victoire : victorySprite (texture victory_background.jpg).

Message de réussite : endMessage (ex. « FELICITATIONS PRINCESSE! » et le numéro du niveau).

Menu d'options : si victoire, options peuvent inclure « Rejouer », « Niveau Suivant », « Quitter ».

Navigation et comportement :

Même interaction clavier que pour l'écran défaite.

« Niveau Suivant » incrémente currentLevel et appelle resetLevel() si disponible.

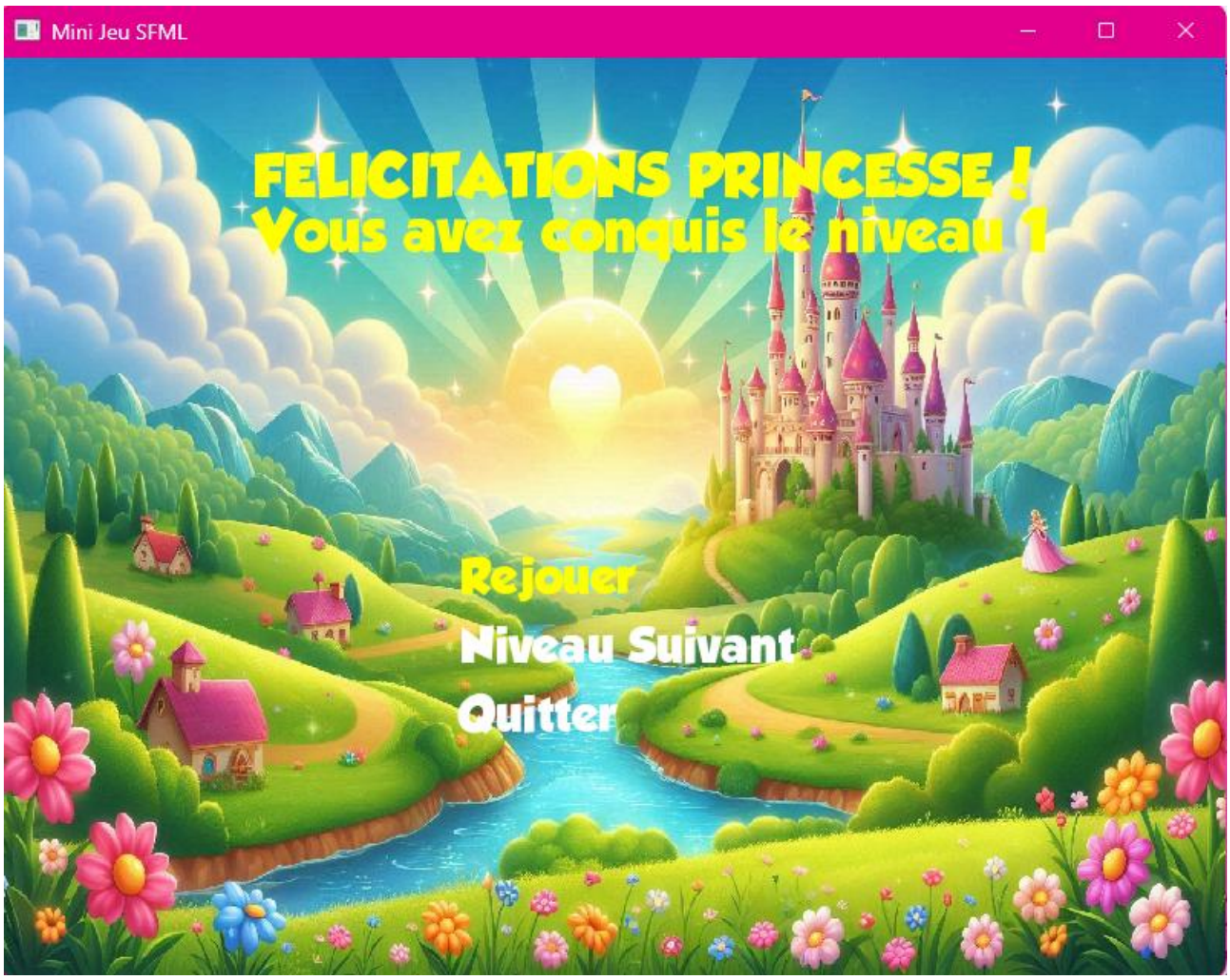

Figure 8 *: Interface vectory {Rejouer ou Niveau suivant ou Quitter}*

7. Description des mécanismes clés

7.1. Boucle de Jeu

La boucle de jeu est gérée par la méthode Game::run() qui exécute, tant que la fenêtre est ouverte, trois étapes à chaque tour : gérer les événements (handleEvents()), mettre à jour la logique (update()), puis dessiner la scène (render()).

La fenêtre est limitée en fréquence d'images par window.setFramerateLimit(90);. Cela stabilise le nombre d'updates par seconde sur la machine de test.

Le projet utilise un sf::Clock pour certaines opérations temporelles (par exemple pour décrémenter le temps de niveau : on vérifie if (clock.getElapsedTime().asSeconds() >= 1.0f) puis on décrémente timeRemaining et on clock.restart()).

7.2. Physique du saut

- L'état saut/accroupi est géré dans Player par des flags isJumping et isDucking.
- Le saut est simulé par une vitesse verticale verticalVelocity mise à jour en additionnant une incrémentation fixe (dans le code actuel verticalVelocity += 1.f; chaque update quand on est en

l'air) et en appliquant sprite.move({0.f, verticalVelocity});. Quand la position Y du sprite dépasse groundY, l'état de saut est fini et la position est réinitialisée au sol.

- Le départ du saut (impulsion initiale) est déclenché par verticalVelocity = -20.f; au moment où le joueur appuie sur la touche Up.
- La gravité est approximée par l'ajout constant de +1.f à verticalVelocity par frame.

7.3. Hitbox Dynamique et précision des collisions

Hitbox (sf::FloatRect) calculée depuis la position et la taille actuelle du sprite :

Ex : réduire la boîte de collision pour correspondre mieux au sprite visuel (marge).

Hitbox dynamique les textures et la position du sprite du joueur changent selon l'état (run/jump/duck). Comme la méthode getBounds() renvoie sprite.getGlobalBounds(), la hitbox suit automatiquement le sprite (taille/position).

Collision detection :

AABB simple : if (player.getBounds().intersects(obstacle.getBounds())) → collision.

Conséquences d'une collision :

diminuer les vies, déclencher animation hurt, éventuellement knockback.

si invincible flag true → ignorer collisions.

- Les collisions sont testées en comparant les bounding boxes fournies par SFML :
  - sf::FloatRect playerBounds = player.getBounds();
  - sf::FloatRect obsBounds = obstacle->getBounds();
  - playerBounds.findIntersection(obsBounds) : pour détecter une intersection et gérer le dégât.
- Par défaut, Obstacle::getBounds() renvoie également sprite.getGlobalBounds().

8. Références

SFML documentation officielle v3.0.2 : https://www.sfml-dev.org/download/sfml/3.0.2/

Les images utiliser sont générer par AI : https://www.bing.com/images/create/ai-image-generator?FORM=IRPGEN ,

Et convertir en png (palace.png) et en jpg pour les backgrounds (ex. background_menu.jpg)

Et nous avons les redimensionnées pour obtenir une lisibilité parfaite pour chaque image.

Font d'écriture GROBOLD.ttf : https://www.dafont.com/fr/grobold.font

9. Conclusion

Ce mini-projet met en œuvre un petit jeu de course/plateforme en C++ en s'appuyant sur la bibliothèque SFML et CMake pour la construction. L'objectif principal était de concevoir une

architecture claire, extensible et robuste pour gérer le joueur, les obstacles et la boucle de jeu, tout en respectant les principes de la programmation orientée objet. Le résultat est une application fonctionnelle qui illustre la séparation des responsabilités entre les composants (affichage, logique de jeu, gestion des ressources) et permet d'ajouter facilement de nouveaux types d'entités.

D'un point de vue technique, le projet a réalisé les fonctionnalités essentielles : boucle de jeu (event/update/render), gestion du joueur (mouvement, saut, vie, invulnérabilité), génération d'obstacles, détection de collisions et menus. L'utilisation d'une interface Entity et d'un stockage polymorphique (std::unique_ptr<Entity>) a permis d'encapsuler le comportement commun et de préparer le terrain pour étendre le jeu (nouveaux obstacles, éléments décoratifs, etc.). La gestion des ressources (textures, polices) a été centralisée via des utilitaires et des caches pour éviter les rechargements inutiles.

Sur le plan du génie logiciel, le projet illustre bien :

- l'héritage et le polymorphisme (contrat Entity / implémentation Obstacle),
- l'encapsulation (membres privés et interfaces publiques claires pour Player, Obstacle, Menu),
- la gestion de la mémoire moderne en C++ (RAII via SFML et smart pointers pour les entités), ce qui réduit les fuites et clarifie la propriété des objets.

Le travail a aussi permis d'identifier des améliorations possibles : la physique du saut est actuellement implémentée avec des incréments par frame et dépend donc du framerate ; l'introduction d'un calcul basé sur le delta-time rendrait la simulation indépendante de la fréquence d'images. De même, la précision des collisions pourrait être améliorée (hitboxes personnalisées, hitboxes composites ou tests per-pixel) pour réduire les faux positifs et affiner le gameplay.

Enfin, le projet fournit une base stable et modulaire pour des extensions futures : niveaux supplémentaires, système de scores, gestionnaire de ressources plus riche, animations et sons, ou encore un moteur de physique plus précis. Ce mini-projet constitue ainsi une bonne démonstration pratique des concepts fondamentaux du développement de jeux 2D en C++ et fournit un squelette propre pour continuer à expérimenter et améliorer les mécanismes de gameplay.

Nous tenons à vous exprimer notre profonde gratitude pour votre encadrement durant la réalisation de ce mini-projet. Vos retours pertinents, votre patience et votre disponibilité nous ont beaucoup aidées à progresser et à mener le projet à bien. Merci sincèrement pour le temps et l'énergie que vous nous avez consacrés.

Bien cordialement,

NAJWA IMMI

GHIZLAN HMAMA